JSON et jeton CSRF

Complément de la séance 1 (bonus) – version étudiant

Avoir terminer le TP de la séance 1
Environnement : XAMPP, VSCodium

Présentation du complément

À la fin de la séance 1, le bonus de l'exercice 6 vous proposait d'enregistrer les inscriptions dans un fichier **JSON**, de refuser les doublons et d'ajouter un **jeton CSRF**.
Ce complément reprend ces trois idées pas à pas : vous découvrirez le format JSON, vous l'utiliserez pour conserver les inscriptions, puis vous **attaquerez votre propre application** avant de la protéger avec un jeton CSRF.

Durée indicative : 1 heure, en autonomie.

Compétences visées

- Lire et écrire du JSON en PHP (`json_encode`, `json_decode`) et gérer ses erreurs.
- Enregistrer des données dans un fichier sans les perdre quand plusieurs visiteurs écrivent en même temps (`flock`).
- Expliquer une attaque **CSRF** et pourquoi le navigateur la rend possible.
- Protéger les formulaires avec un **jeton CSRF** (`random_bytes`, `hash_equals`).
- Fournir des données au format JSON à du JavaScript (`fetch`).

Prérequis : la séance 1 terminée (`fonctions.php` et `inscription.php` de l'exercice 6), sans base de données : la séance 2 remplacera le fichier JSON par MariaDB.

Déroulé conseillé

Étape	Durée	Contenu
Partie 1 et exercices A et B	30 min	JSON, enregistrement dans un fichier, doublons
Partie 2 et exercices C, D, E	30 min	Mode organisateur, attaque CSRF, jeton
Exercice F	bonus	Une API JSON consommée par JavaScript

Arborescence du projet

```
donnees/
└── inscriptions.json     fichier de données : HORS du dossier tp-php
tp-php/
├── fonctions.php         h(), champ()… (séance 1)
├── Inscription.php       enum Filiere + classe readonly Inscription
├── stockage.php          lecture et écriture du fichier JSON
├── csrf.php              jeton CSRF
├── inscription.php       formulaire d'inscription (modifié)
├── liste.php             liste des inscrits
├── organisateur.php      « mode organisateur » (jouet)
├── supprimer.php         suppression d'une inscription
├── api_inscriptions.php  données au format JSON
├── participants.html     page JavaScript qui lit l'API
└── json_demo.php         découverte de JSON
pirate/
└── evil.html             page de l'« attaquant » (autre dossier)
```

Partie 1 – JSON

Un format que vous connaissez déjà

JSON (*JavaScript Object Notation*) est un format texte pour échanger des données. Vous l'avez rencontré en JavaScript avec `JSON.stringify()` et `JSON.parse()` ; PHP a les fonctions équivalentes.

```
{
  "nom": "Ada",
  "age": 36,
  "actif": true,
  "diplome": null,
  "langages": ["PHP", "C"],
  "adresse": { "ville": "Lyon" }
}
```

Règles de syntaxe, plus strictes qu'en JavaScript :

- les clés et les chaînes sont entre **guillemets doubles** (jamais simples) ;
- **pas de virgule** après le dernier élément ;
- pas de commentaires ;
- valeurs possibles : chaîne, nombre, `true` / `false`, `null`, tableau `[...]`, objet `{...}`.

En PHP	En JSON
Tableau indexé `['PHP', 'C']`	Tableau `["PHP","C"]`
Tableau associatif `['nom' => 'Ada']`	Objet `{"nom":"Ada"}`
`int`, `float`	Nombre
`string`	Chaîne
`bool`, `null`	`true` / `false`, `null`
Objet : ses propriétés **publiques**	Objet
`enum` « backed » (`Filiere::Info`)	Sa valeur (`"INFO"`)

Un tableau PHP vide `[]` devient toujours `[]` en JSON, jamais `{}`.

`json_encode` et `json_decode`

```
<?php
declare(strict_types=1);

$etudiant = [
    'nom'      => 'Zoé Martin',
    'age'      => 19,
    'note'     => 14.5,
    'actif'    => true,
    'diplome'  => null,
    'langages' => ['PHP', 'JavaScript'],
];

echo json_encode($etudiant), PHP_EOL;
echo json_encode($etudiant, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE), PHP_EOL;
```

```
$json = '{"nom":"Ada","age":36,"langages":["PHP","C"],"adresse":{"ville":"Lyon"}}';

$tableau = json_decode($json, true);        // true : on obtient des tableaux
associatifs
echo $tableau['adresse']['ville'], PHP_EOL;

$objet = json_decode($json);                // sans true : on obtient des objets
(stdClass)
echo $objet->adresse->ville, PHP_EOL;

try {
    json_decode('{"nom": "Ada",}', true, 512, JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    echo 'JSON invalide : ', $e->getMessage(), PHP_EOL;
}

var_dump(json_decode('{"nom": "Ada",}', true));   // sans JSON_THROW_ON_ERROR : null,
sans explication
echo json_last_error_msg(), PHP_EOL;
```

Option	Effet
`JSON_PRETTY_PRINT`	Mise en forme lisible (retours à la ligne, indentation)
`JSON_UNESCAPED_UNICODE`	Garde les accents tels quels, au lieu de `\u00e9`
`JSON_THROW_ON_ERROR`	Lève une `JsonException` au lieu de renvoyer `null` ou `false` sans explication
`true` (2e argument de `json_decode`)	Produit des tableaux associatifs plutôt que des objets `stdClass`

> **Toujours utiliser `JSON_THROW_ON_ERROR`.** Sinon, un JSON invalide donne simplement `null`, qu'on confond facilement avec un JSON qui contient `null`.

Un JSON reçu de l'extérieur reste une donnée **non fiable** : son contenu doit être validé comme n'importe quelle saisie (séance 1), et échappé avant affichage.

Enregistrer dans un fichier : les pièges

Écrire `file_put_contents('inscriptions.json', json_encode($liste))` fonctionne… tant qu'il n'y a qu'un visiteur. Quatre pièges à connaître.

1. La perte de données. Pour ajouter une inscription, il faut *lire* la liste, *ajouter* un élément, puis *réécrire* le fichier. Si deux visiteurs le font en même temps, chacun lit la même liste de départ et le second écrase l'ajout du premier. Dans un test de 30 envois simultanés, la version sans protection n'a conservé qu'une petite partie des inscriptions.

2. `LOCK_EX` ne suffit pas. `file_put_contents(..., LOCK_EX)` verrouille uniquement l'*écriture* : la lecture qui précède reste non protégée. Il faut un verrou qui couvre **tout le cycle** lire, modifier, écrire. C'est le rôle de `flock()` sur un fichier ouvert avec `fopen()`.

3. L'emplacement du fichier. Le serveur de PHP sert tout ce qui se trouve dans le dossier du projet : un fichier `inscriptions.json` placé là serait lisible par n'importe qui à l'adresse `http://localhost:8000/inscriptions.json`, avec les adresses e-mail de tous les inscrits. On le place donc **en dehors** du dossier du projet (dossier voisin `donnees`).

> Avec l'option B de la séance 1 (Apache, dossier dans `C:\xampp\htdocs`), le dossier parent `htdocs` est lui aussi servi par le navigateur : remontez d'un niveau de plus avec `dirname(__DIR__, 2)`.

4. Les dates. `date('c')` donne la date au format ISO 8601 (`2026-10-14T10:00:00+02:00`). Si le décalage affiché ne correspond pas à votre fuseau horaire, ajoutez `date_default_timezone_set('Europe/Paris');` au début du script.

Le modèle de données

Une inscription est un petit objet typé. L'`enum` et la classe `readonly` de la séance 1 se transforment sans effort en JSON (l'`enum` devient sa valeur) :

```
<?php
declare(strict_types=1);

enum Filiere: string
{
    case Info  = 'INFO';
    case Math  = 'MATH';
    case Phys  = 'PHYS';
    case Autre = 'AUTRE';
}

final readonly class Inscription
{
    /** @param string[] $langages */
    public function __construct(
        public int $id,
        public string $date,
        public string $nom,
        public string $email,
        public int $age,
        public Filiere $filiere,
        public array $langages,
        public string $message,
    ) {}

    /** @param array<string, mixed> $d */
    public static function depuisTableau(array $d): self
    {
        return new self(
            id: (int) $d['id'],
            date: (string) $d['date'],
            nom: (string) $d['nom'],
            email: (string) $d['email'],
            age: (int) $d['age'],
            filiere: Filiere::tryFrom((string) $d['filiere']) ?? Filiere::Autre,
            langages: array_values(array_filter((array) ($d['langages'] ?? []),
'is_string')),
            message: (string) ($d['message'] ?? ''),
        );
    }
}
```

Le module de stockage

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/Inscription.php';

/** Le fichier est placé HORS du dossier du projet : le serveur web ne peut pas le
servir. */
const FICHIER_INSCRIPTIONS = __DIR__ . '/../donnees/inscriptions.json';

/** @return Inscription[] */
function decoderInscriptions(string $json): array
{
    if (trim($json) === '') {
        return [];
    }
    $donnees = json_decode($json, true, 512, JSON_THROW_ON_ERROR);
    return array_map(fn(array $d): Inscription => Inscription::depuisTableau($d),
$donnees);
}

/** @return Inscription[] */
function lireInscriptions(): array
{
    if (!is_file(FICHIER_INSCRIPTIONS)) {
        return [];
    }
    $fichier = fopen(FICHIER_INSCRIPTIONS, 'r');
    if ($fichier === false) {
        throw new RuntimeException('Fichier de données illisible.');
    }
    try {
        flock($fichier, LOCK_SH);                   // verrou partagé : on lit pendant
qu'on n'écrit pas
        $contenu = (string) stream_get_contents($fichier);
        flock($fichier, LOCK_UN);
    } finally {
        fclose($fichier);
    }
    return decoderInscriptions($contenu);
}

/**
 * Lit la liste, la transmet à $traitement, puis réécrit la liste renvoyée.
 * Le verrou exclusif couvre TOUT le cycle lire - modifier - écrire :
 * deux requêtes simultanées ne peuvent pas s'écraser mutuellement.
 *
 * @param callable(Inscription[]): Inscription[] $traitement
 */
function modifierInscriptions(callable $traitement): void
{
    $dossier = dirname(FICHIER_INSCRIPTIONS);
    if (!is_dir($dossier) && !mkdir($dossier, 0775, true) && !is_dir($dossier)) {
        throw new RuntimeException('Dossier de données impossible à créer.');
    }

    $fichier = fopen(FICHIER_INSCRIPTIONS, 'c+');   // c+ : crée si absent, lit et
écrit, sans vider
    if ($fichier === false) {
        throw new RuntimeException('Fichier de données inaccessible.');
    }
```

```
    try {
        if (!flock($fichier, LOCK_EX)) {
            throw new RuntimeException('Verrou impossible à obtenir.');
        }
        $liste = decoderInscriptions((string) stream_get_contents($fichier));
        $liste = $traitement($liste);

        $json = json_encode($liste, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE |
JSON_THROW_ON_ERROR);
        ftruncate($fichier, 0);
        rewind($fichier);
        fwrite($fichier, $json);
        fflush($fichier);
        flock($fichier, LOCK_UN);
    } finally {
        fclose($fichier);
    }
}

/** Renvoie l'inscription créée, ou null si l'adresse e-mail est déjà inscrite. */
function ajouterInscription(array $donnees): ?Inscription
{
    $creee = null;

    modifierInscriptions(function (array $liste) use ($donnees, &$creee): array {
        foreach ($liste as $existante) {
            if (mb_strtolower($existante->email) === mb_strtolower($donnees['email']))
{
                return $liste;                      // doublon : rien ne change
            }
        }
        $id = $liste === [] ? 1 : max(array_map(fn(Inscription $i): int => $i->id,
$liste)) + 1;
        $creee = Inscription::depuisTableau($donnees + ['id' => $id, 'date' =>
date('c')]);
        $liste[] = $creee;
        return $liste;
    });

    return $creee;
}

function supprimerInscription(int $id): bool
{
    $supprimee = false;

    modifierInscriptions(function (array $liste) use ($id, &$supprimee): array {
        $reste     = array_values(array_filter($liste, fn(Inscription $i): bool => $i-
>id !== $id));
        $supprimee = count($reste) !== count($liste);
        return $reste;
    });

    return $supprimee;
}
```

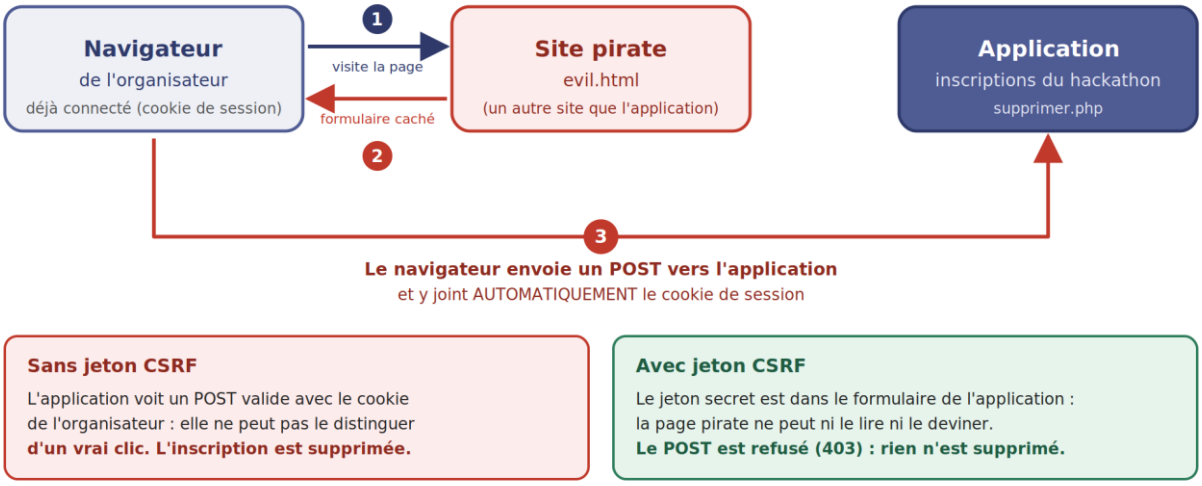
Points à comprendre :

- `fopen(..., 'c+')` crée le fichier s'il n'existe pas, permet de lire **et** d'écrire, et ne le vide pas à l'ouverture.
- `modifierInscriptions()` prend en paramètre une **fonction anonyme** qui reçoit la liste et renvoie la liste modifiée. Le verrou exclusif est posé avant la lecture et relâché après l'écriture.
- La vérification du **doublon d'e-mail** se fait *à l'intérieur* du verrou : deux inscriptions simultanées avec la même adresse ne peuvent pas passer toutes les deux.
- `use (..., &$creee)` permet à la fonction anonyme de renvoyer une information (l'inscription créée) à la fonction qui l'a appelée.
- Le bloc `try ... finally` garantit que le fichier est refermé même en cas d'exception.

Partie 2 – Le jeton CSRF

Le principe de l'attaque

CSRF (*Cross-Site Request Forgery*, « falsification de requête inter-sites ») exploite un comportement normal des navigateurs : quand une page envoie une requête vers un site, le navigateur y joint **automatiquement les cookies** de ce site, **quelle que soit la page qui a déclenché la requête**.



Attaque CSRF : une page pirate fait envoyer un POST par le navigateur d'une victime connectée

Imaginez un organisateur connecté à l'application. Il ouvre, dans un autre onglet, une page piégée (reçue par message, par exemple). Cette page contient un formulaire invisible qui vise `supprimer.php` et s'envoie tout seul. Le navigateur joint le cookie de session de l'organisateur : pour l'application, la requête est indiscernable d'un vrai clic. **L'attaquant n'a jamais eu besoin de connaître le moindre mot de passe.**

Trois conditions rendent l'attaque possible :

1. une **action sensible** peut être déclenchée par une requête que l'attaquant sait fabriquer (`POST id=1`) ;
2. l'identité de l'utilisateur est transmise **automatiquement** (cookie de session) ;
3. la requête ne contient **aucun secret** que seul le vrai formulaire connaît.

La parade : le jeton CSRF

Le **jeton** (*token*) est une valeur aléatoire, propre à la session, qui joue le rôle de ce secret :

1. le serveur génère un jeton imprévisible et le **garde en session** ;
2. il l'insère dans chaque formulaire, dans un **champ caché** ;
3. à la réception d'un POST, il **compare** le jeton reçu à celui de la session et refuse la requête s'ils diffèrent.

La page pirate ne peut ni deviner le jeton (il est aléatoire), ni le lire dans le formulaire de l'application (le navigateur interdit à une page d'un site de lire le contenu d'un autre site : c'est la *politique de même origine*).

```php
<?php
declare(strict_types=1);

/** Le jeton est propre à la session : il faut avoir appelé session_start() avant. */
function csrfToken(): string
{
    if (session_status() !== PHP_SESSION_ACTIVE) {
        throw new LogicException('csrfToken() exige une session démarrée.');
    }
    if (!isset($_SESSION['csrf']) || !is_string($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(32));    // 64 caractères
hexadécimaux, imprévisibles
    }
    return $_SESSION['csrf'];
}

/** Champ caché à placer dans chaque formulaire POST. */
function csrfChamp(): string
{
    return '<input type="hidden" name="csrf" value="' . h(csrfToken()) . '">';
}

function csrfValide(): bool
{
    $recu = $_POST['csrf'] ?? '';
    return is_string($recu) && hash_equals(csrfToken(), $recu);   // comparaison à
temps constant
}

/** À appeler au début du traitement de tout POST : refuse la requête si le jeton est
absent ou faux. */
function csrfExiger(): void
{
    if (!csrfValide()) {
        http_response_code(403);
        exit('Requête refusée : jeton CSRF invalide ou absent.');
    }
}
```

Choix	Pourquoi
`random_bytes(32)`	Source aléatoire sûre pour la cryptographie. Jamais `rand()`, `uniqid()` ni `md5(time())`, qui sont prévisibles
`hash_equals()`	Comparaison à temps constant : ne révèle pas, par son temps de réponse, combien de caractères sont corrects
Jeton en session	Il est lié au visiteur : le jeton d'un autre visiteur est refusé
Contrôle des types (`is_string`)	Un attaquant peut envoyer `csrf[]=a` (tableau)

Règles à retenir

- **Les actions qui modifient des données se font en POST**, jamais en GET : un lien ou une balise `<img src="supprimer.php?id=1">` suffirait sinon à déclencher l'action, sans même de formulaire.
- **Chaque formulaire POST** contient le jeton, et **chaque traitement** le vérifie avant toute autre chose.
- Le jeton ne se met **jamais dans l'URL** (elle apparaît dans l'historique et les journaux).
- Le cookie de session peut aussi porter l'attribut `SameSite=Lax` (complément sur les sessions) : le navigateur n'envoie alors pas le cookie dans de nombreuses requêtes venant d'un autre site. C'est une protection **complémentaire**, qui ne remplace pas le jeton : certaines requêtes passent quand même, et les navigateurs diffèrent.
- Le jeton ne protège pas d'une faille **XSS** : un script injecté dans la page peut lire le champ caché. D'où l'importance d'échapper toutes les sorties avec `h()`.

> **À propos de la démonstration du TP.** `localhost:8000` (l'application) et `localhost:9000` (le « site pirate ») ont des ports différents, mais les navigateurs considèrent que ce sont deux **origines** distinctes du **même site** : le cookie est donc envoyé, même avec `SameSite=Lax`. Avec un vrai site pirate, sur un autre domaine, la protection `SameSite` pourrait déjà bloquer cette attaque précise, selon le navigateur et la configuration du cookie ; le jeton reste indispensable pour couvrir tous les cas.

TP complémentaire

Exercice A – Découvrir JSON en ligne de commande (10 min)

Objectif : manipuler `json_encode` et `json_decode`.

- Créer `json_demo.php` avec le code de la partie 1, puis l'exécuter avec `php json_demo.php`.
- Modifier le script pour répondre aux questions suivantes :
  - Que donne `json_encode` **sans** `JSON_UNESCAPED_UNICODE` pour « Zoé » ? Le résultat est-il du JSON valide ?
  - Que renvoie `json_decode('{"nom": \'Ada\'}', true)` (guillemets simples) ? Et avec `JSON_THROW_ON_ERROR` ?
  - Comment accéder à `ville` avec `json_decode($json)` (sans `true`) et avec `json_decode($json, true)` ?
  - Que donne `json_encode([])` ? Et `json_encode(new stdClass)` ?

À vérifier

- ☐ Vous savez expliquer pourquoi un JSON avec une virgule finale est invalide.
- ☐ Vous savez expliquer la différence entre `null` renvoyé par erreur et `null` comme valeur JSON valide.

Exercice B – Enregistrer les inscriptions (20 min)

Objectif : conserver les inscriptions dans un fichier JSON, sans doublon d'e-mail et sans perte de données.

1. Créer `Inscription.php` et `stockage.php` à partir des modèles de la partie 1. Dans `inscription.php`, **supprimer** la déclaration de l'`enum Filiere` (elle est désormais dans `Inscription.php`) et ajouter `require_once __DIR__ . '/stockage.php';`.
2. Dans `inscription.php`, remplacer le traitement d'un formulaire valide (le bloc `if ($erreurs === []) { ... }`) par :

```php
if ($erreurs === []) {
    try {
        $inscription = ajouterInscription([
            'nom'       => $nom,
            'email'     => $email,
            'age'       => (int) $age,
            'filiere'   => $filiere,
            'langages'  => $langages,
            'message'   => $message,
        ]);

        if ($inscription === null) {
            $erreurs['email'] = 'Cette adresse e-mail est déjà inscrite.';
        } else {
            $_SESSION['confirmation'] = "Merci $nom, votre inscription est
enregistrée.";
            header('Location: inscription.php');
            exit;
        }
    } catch (JsonException | RuntimeException $e) {
        error_log('Stockage : ' . $e->getMessage());
        $erreurs['general'] = 'Enregistrement impossible pour le moment.';
    }
}
```

3. Afficher l'erreur `general` au-dessus du formulaire : `<?php afficherErreur($erreurs, 'general'); ?>`.
4. Faire deux inscriptions valides, puis ouvrir le fichier `donnees/inscriptions.json` dans VSCodium.
5. Faire une troisième inscription avec la même adresse e-mail, en changeant la casse (`ADA@example.org`).
6. **Expérience :** déplacer temporairement le fichier JSON dans `tp-php/donnees/` (en changeant la constante `FICHIER_INSCRIPTIONS`) et ouvrir `http://localhost:8000/donnees/inscriptions.json` dans le navigateur. Remettre ensuite le fichier à sa place.

À vérifier

- ☐ Le fichier JSON est valide, lisible, avec les accents préservés et un identifiant par inscription.
- ☐ La valeur de `filiere` est `"INFO"` (et non un objet) : pourquoi ?
- ☐ L'adresse e-mail déjà utilisée est refusée, même avec une casse différente.
- ☐ Lors de l'expérience, le fichier est lisible par n'importe qui : c'est ce qu'on évite en le plaçant hors du dossier du projet.
- ☐ Question : pourquoi la vérification du doublon est-elle faite *dans* `modifierInscriptions()` et non avant ?

Exercice C – Le « mode organisateur » (15 min)

Objectif : disposer d'une action sensible, réservée à une personne identifiée par sa session, qu'on pourra ensuite attaquer.

La séance 1 n'a pas de véritable connexion. Pour illustrer l'attaque, on simule un « mode organisateur » : un code unique à saisir, qui marque la session. **Ce n'est pas une vraie authentification** : la séance 2 en construira une avec une base de données et des mots de passe hachés.

Les trois fichiers suivants vous sont **fournis** : copiez-les, puis lisez-les attentivement (ils ne contiennent pas encore de protection CSRF).

`liste.php` :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';
require_once __DIR__ . '/stockage.php';

session_start();

$organisateur = ($_SESSION['organisateur'] ?? false) === true;
$inscriptions = [];
$erreur       = null;

try {
    $inscriptions = lireInscriptions();
} catch (JsonException | RuntimeException $e) {
    error_log('Lecture : ' . $e->getMessage());
    $erreur = 'Données momentanément indisponibles.';
}

$flash = $_SESSION['flash'] ?? null;
unset($_SESSION['flash']);
?>
```

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Liste des inscrits</title></head>
<body>
  <h1>Inscrits (<?= count($inscriptions) ?>)</h1>

  <?php if (is_string($flash)): ?>
    <p style="color:#060"><?= h($flash) ?></p>
  <?php endif; ?>
  <?php if ($erreur !== null): ?>
    <p style="color:#b00"><?= h($erreur) ?></p>
  <?php endif; ?>

  <table border="1" cellpadding="6">
    <tr>
      <th>Id</th><th>Nom</th><th>Filière</th><th>Langages</th>
      <?php if ($organisateur): ?><th>E-mail</th><th>Action</th><?php endif; ?>
    </tr>
    <?php foreach ($inscriptions as $i): ?>
      <tr>
        <td><?= $i->id ?></td>
        <td><?= h($i->nom) ?></td>
        <td><?= h($i->filiere->value) ?></td>
        <td><?= h(implode(', ', $i->langages)) ?></td>
        <?php if ($organisateur): ?>
          <td><?= h($i->email) ?></td>
          <td>
            <form method="post" action="supprimer.php">
              <input type="hidden" name="id" value="<?= $i->id ?>">
              <button type="submit">Supprimer</button>
            </form>
          </td>
        <?php endif; ?>
      </tr>
    <?php endforeach; ?>
  </table>

  <p>
    <a href="inscription.php">Inscription</a> ·
    <a href="organisateur.php"><?= $organisateur ? 'Mode organisateur (actif)' : 'Mode
organisateur' ?></a>
  </p>
</body>
</html>
```

`organisateur.php` :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';

session_start();

/** Code d'accès du TP. Une vraie application utiliserait une authentification (séance
2). */
const CODE_ORGANISATEUR = 'hackathon2026';

$erreur = null;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
    if (champ('action') === 'quitter') {
        unset($_SESSION['organisateur']);
        header('Location: liste.php');
        exit;
    }

    if (hash_equals(CODE_ORGANISATEUR, champ('code'))) {
        session_regenerate_id(true);
        $_SESSION['organisateur'] = true;
        header('Location: liste.php');
        exit;
    }
    $erreur = 'Code incorrect.';
}

$organisateur = ($_SESSION['organisateur'] ?? false) === true;
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Mode organisateur</title></head>
<body>
  <h1>Mode organisateur</h1>

  <?php if ($erreur !== null): ?>
    <p style="color:#b00"><?= h($erreur) ?></p>
  <?php endif; ?>

  <?php if ($organisateur): ?>
    <p>Vous êtes en mode organisateur.</p>
    <form method="post" action="organisateur.php">
      <input type="hidden" name="action" value="quitter">
      <button type="submit">Quitter le mode organisateur</button>
    </form>
  <?php else: ?>
    <form method="post" action="organisateur.php">
      <label>Code organisateur <input type="password" name="code"></label>
      <button type="submit">Entrer</button>
    </form>
  <?php endif; ?>

  <p><a href="liste.php">Retour à la liste</a></p>
</body>
</html>
```

`supprimer.php` :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';
require_once __DIR__ . '/stockage.php';

session_start();

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    header('Allow: POST');
    exit('Méthode non autorisée.');
}
if (($_SESSION['organisateur'] ?? false) !== true) {
```

```
    http_response_code(403);
    exit('Accès réservé à l\'organisateur.');
}

$id = filter_var(champ('id'), FILTER_VALIDATE_INT);
if ($id === false) {
    http_response_code(400);
    exit('Identifiant invalide.');
}

try {
    $supprimee = supprimerInscription($id);
    $_SESSION['flash'] = $supprimee ? "Inscription $id supprimée." : "Inscription $id
introuvable.";
} catch (JsonException | RuntimeException $e) {
    error_log('Suppression : ' . $e->getMessage());
    $_SESSION['flash'] = 'Suppression impossible pour le moment.';
}

header('Location: liste.php');
exit;
```

À vérifier

- ☐ La liste publique ne montre ni les e-mails ni les boutons de suppression.
- ☐ Après saisie du code `hackathon2026` dans `organisateur.php`, la liste affiche les e-mails et un bouton **Supprimer** par ligne ; le bouton fonctionne.
- ☐ Ouvrir `http://localhost:8000/supprimer.php` directement dans le navigateur renvoie « Méthode non autorisée » (405).
- ☐ Depuis une fenêtre de navigation privée (pas organisateur), un POST vers `supprimer.php` est refusé (403).

Exercice D – L'attaque (10 min)

Objectif : constater qu'une page extérieure peut faire supprimer une inscription.

> Cette page vise uniquement **votre propre application, en local**. Attaquer un site sans autorisation est interdit.

1. Créer un dossier `pirate` (à côté de `tp-php`, pas dedans) contenant `evil.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Vous avez gagné !</title></head>
<body>
  <h1>Félicitations, vous avez gagné un voyage !</h1>
  <p>Chargement de votre cadeau…</p>

  <!-- Formulaire invisible : il vise l'application de l'organisateur et s'envoie tout
seul. -->
  <form id="piege" action="http://localhost:8000/supprimer.php" method="post">
    <input type="hidden" name="id" value="1">
  </form>
  <script>document.getElementById('piege').submit();</script>
</body>
</html>
```

2. Dans un **second terminal**, se placer dans le dossier `pirate` et lancer un deuxième serveur : `php -S localhost:9000`.
3. Dans le navigateur, passer en mode organisateur sur `http://localhost:8000/organisateur.php`, puis, **dans le même navigateur**, ouvrir `http://localhost:9000/evil.html`.
4. Revenir sur `http://localhost:8000/liste.php`.
5. Recommencer dans une fenêtre de navigation privée (sans mode organisateur).

À vérifier

- ☐ En mode organisateur, l'inscription d'identifiant 1 a disparu, sans que vous ayez cliqué sur un bouton.
- ☐ Sans mode organisateur, la page pirate ne produit aucun effet (l'action est refusée).
- ☐ Question : pourquoi le navigateur envoie-t-il le cookie de session alors que la requête part de la page `localhost:9000` ?
- ☐ Question : l'attaquant connaît-il le code organisateur ? En a-t-il besoin ?
- ☐ Question : que se passerait-il si la suppression se faisait par un lien en GET (`supprimer.php?id=1`) ?

Exercice E – Le jeton CSRF (15 min)

Objectif : bloquer l'attaque sans gêner les vrais utilisateurs.

1. Créer `csrf.php` à partir du modèle de la partie 2.
2. Protéger les quatre fichiers qui reçoivent ou affichent un formulaire POST :

Fichier	Modification
`inscription.php`	Ajouter `require_once __DIR__ . '/csrf.php';`, appeler `csrfExiger();` au début du bloc `if ($_SERVER['REQUEST_METHOD'] === 'POST')`, et ajouter `<?= csrfChamp() ?>` dans le formulaire
`liste.php`	Ajouter le `require_once` et `<?= csrfChamp() ?>` dans le formulaire du bouton **Supprimer**
`organisateur.php`	Ajouter le `require_once`, `csrfExiger();` au début du bloc POST, et `<?= csrfChamp() ?>` dans **les deux** formulaires
`supprimer.php`	Ajouter le `require_once` et `csrfExiger();` juste après la vérification du mode organisateur

3. Relancer l'attaque de l'exercice D.
4. Vérifier que les formulaires légitimes fonctionnent toujours.
5. Avec les outils de développement (`F12`), modifier la valeur du champ caché `csrf` d'un formulaire, puis l'envoyer.

À vérifier

- ☐ L'attaque de l'exercice D est maintenant refusée : message « jeton CSRF invalide ou absent » (403), et l'inscription reste en place.
- ☐ Inscription, mode organisateur et suppression légitime fonctionnent normalement.
- ☐ Un jeton modifié à la main est refusé.
- ☐ Dans le code source de la page, le champ caché contient 64 caractères hexadécimaux.
- ☐ Question : pourquoi la page pirate ne peut-elle pas simplement inclure le jeton dans son formulaire ?
- ☐ Question : pourquoi `hash_equals` plutôt que `===`, et `random_bytes` plutôt que `rand` ?

Pour aller plus loin (bonus)

- Régénérer le jeton à chaque changement de privilège (entrée dans le mode organisateur).
- Utiliser un jeton **par formulaire** qui ne sert qu'une fois.
- Ajouter l'attribut `SameSite=Lax` au cookie de session (voir le complément sur les sessions).

Exercice F (bonus) – Une API JSON consommée par JavaScript (15 min)

Objectif : fournir les inscriptions au format JSON à une page JavaScript.

`api_inscriptions.php` renvoie la liste publique (sans les e-mails) :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/stockage.php';

header('Content-Type: application/json; charset=utf-8');

try {
    // On n'expose que ce qui est public : jamais les adresses e-mail.
    $publiques = array_map(
        fn(Inscription $i): array => [
            'id'       => $i->id,
            'nom'      => $i->nom,
            'filiere'  => $i->filiere->value,
            'langages' => $i->langages,
        ],
        lireInscriptions()
    );

    echo json_encode(
        ['total' => count($publiques), 'inscriptions' => $publiques],
        JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR
    );
} catch (JsonException | RuntimeException $e) {
    error_log('API : ' . $e->getMessage());
    http_response_code(500);
    echo json_encode(['erreur' => 'Service momentanément indisponible.']);
}
```

`participants.html` la lit avec `fetch` et affiche les participants :

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Participants</title></head>
<body>
  <h1>Participants</h1>
  <p id="total">Chargement…</p>
  <ul id="liste"></ul>

  <script>
    async function charger() {
      const reponse = await fetch('api_inscriptions.php');
      if (!reponse.ok) {
        document.getElementById('total').textContent = 'Service indisponible.';
        return;
      }
      const donnees = await reponse.json();

      document.getElementById('total').textContent = `${donnees.total} inscrit(s)`;
      const elements = donnees.inscriptions.map((i) => {
        const li = document.createElement('li');
        li.textContent = `${i.nom} (${i.filiere}) : ${i.langages.join(', ')}`;   //
textContent, jamais innerHTML
        return li;
      });
      document.getElementById('liste').replaceChildren(...elements);
    }
    charger();
  </script>
</body>
</html>
```

- Ouvrir `http://localhost:8000/api_inscriptions.php` dans le navigateur, puis `http://localhost:8000/participants.html`.
- Faire une inscription dont le nom est `<b>Ada</b>` et recharger `participants.html`.
- Dans `participants.html`, remplacer temporairement `li.textContent = ...` par `li.innerHTML = ...` et recharger. Que se passe-t-il ? Remettre `textContent`.

À vérifier

- ☐ L'API renvoie l'en-tête `Content-Type: application/json` et **aucune adresse e-mail**.
- ☐ Le nom `<b>Ada</b>` est affiché tel quel (en texte) avec `textContent`.
- ☐ Avec `innerHTML`, le nom est interprété comme du HTML : c'est une faille XSS côté JavaScript. Pourquoi `textContent` est-il le bon choix ?

Annexes

A. Aide-mémoire

Besoin	Code
Convertir en JSON	`json_encode($valeur, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR)`
Lire du JSON (tableaux)	`json_decode($texte, true, 512, JSON_THROW_ON_ERROR)`
Capturer une erreur JSON	`catch (JsonException $e)`
Réponse JSON d'une page PHP	`header('Content-Type: application/json; charset=utf-8');`
Ouvrir un fichier (créer si absent)	`fopen($chemin, 'c+')`
Verrouiller / déverrouiller	`flock($f, LOCK_EX)` / `flock($f, LOCK_UN)`
Jeton aléatoire	`bin2hex(random_bytes(32))`
Comparer deux secrets	`hash_equals($attendu, $recu)`
Champ caché du jeton	`<input type="hidden" name="csrf" value="...">`
Refuser une requête	`http_response_code(403); exit('...');`
Lire du JSON en JavaScript	`const donnees = await (await fetch(url)).json();`

B. Erreurs fréquentes et diagnostic

Symptôme	Cause probable	Solution
`JsonException: Syntax error`	Virgule finale, guillemets simples, fichier tronqué	Corriger le JSON ; vérifier le fichier de données
Accents affichés `\u00e9`	`JSON_UNESCAPED_UNICODE` oublié	Ajouter l'option (le JSON reste valide dans les deux cas)
Le formulaire affiche « Enregistrement impossible »	Droits insuffisants ou dossier absent	Vérifier que le dossier voisin `donnees` est créable ; lire le terminal du serveur
Les inscriptions sont dans un fichier inattendu	Chemin relatif au lieu de `__DIR__`	Utiliser `__DIR__ . '/../donnees/...'`
`Cannot declare enum Filiere` ou `Class "Filiere" not found`	Enum déclaré deux fois, ou plus du tout	Le déclarer une seule fois, dans `Inscription.php`, inclus partout
403 « jeton CSRF invalide » sur un formulaire légitime	`csrfChamp()` absent du formulaire, ou cookies bloqués, ou session expirée	Ajouter le champ ; recharger la page pour obtenir un jeton frais
`csrfToken() exige une session démarrée`	`session_start()` oublié ou placé après l'appel	Appeler `session_start()` en tout début de page
L'attaque réussit toujours après l'exercice E	`csrfExiger()` absent de `supprimer.php`	Le placer après la vérification du mode organisateur
La page pirate n'a aucun effet	Serveur 8000 arrêté, mauvaise adresse, ou mode organisateur actif dans un autre navigateur	Vérifier le port ; utiliser le **même** navigateur
`Failed to listen ... Address already in use` (port 9000)	Un serveur occupe déjà le port	`php -S localhost:9001` et adapter l'adresse
La liste est vide alors que le fichier contient des données	`JsonException` à la lecture	Lire le journal du terminal ; vérifier la validité du fichier

C. Pour aller plus loin

- **Formats :** JSON n'est pas le seul format d'échange (CSV, XML, YAML) ; le JSON est le plus courant pour les API web.
- **Validation de JSON :** *JSON Schema* permet de décrire et de vérifier la structure attendue d'un document.
- **CSRF :** variante « double soumission de cookie » pour les applications sans session côté serveur ; cookie `SameSite=Strict` ; vérification des en-têtes `Origin` et `Referer`.
- **Références :** la documentation PHP sur JSON (`php.net/manual/fr/book.json.php`) et le guide de prévention du CSRF de l'OWASP.